SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

VELLORE INSTITUTE OF TECHNOLOGY, VELLORE-14

PROJECT

Design and Implementation of an LL(1) Parser Generator with Automatic FIRST-FOLLOW Computation, Predictive Parsing Table Construction and Syntax-Directed Translation

A dissertation submitted in partial fulfilment of the requirements for the degree of

Master of Technology in Integrated Computer Science & Engineering

by

« Student Name 1 > **24BCE0092**

« Student Name 2 > **24BDE0155**

« Student Name 3 > **24BCE0556**

« Student Name 4 > **24BCI0103**

« Student Name 5 >

24BDS0044

Under the Guidance of

Prof. « Guide Name >

School of Computer Science and Engineering,
VIT, Vellore–14

« Month , « Year >

DECLARATION

I hereby declare that the project entitled **Design and Implementation of an LL(1) Parser Generator with Automatic FIRST-FOLLOW Computation, Predictive Parsing Table Construction and Syntax-Directed Translation** submitted by **Team 4** (registration numbers 24BCE0092, 24BDE0155, 24BCE0556, 24BCI0103 and 24BDS0044) to the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore-14, towards the partial fulfilment of the requirements for the award of the degree of **Master of Technology in Integrated Computer Science & Engineering** is a record of bonafide work carried out by the team under the supervision of **Prof. « Guide Name >**. I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or of any other institute or university.

Signature	**Signature**	**Signature**	**Signature**	**Signature**
Name	**Name**	**Name**	**Name**	**Name**
Reg. No:	**Reg. No:**	**Reg. No:**	**Reg. No:**	**Reg. No:**

CERTIFICATE

The project report entitled **Design and Implementation of an LL(1) Parser Generator with Automatic FIRST-FOLLOW Computation, Predictive Parsing Table Construction and Syntax-Directed Translation** is prepared and submitted by the students « **Student Names with Registration Numbers** >, and has been found satisfactory in terms of scope, quality and presentation as partial fulfilment of the requirements for the award of the degree of **Master of Technology in Integrated Computer Science and Engineering** in Vellore Institute of Technology, Vellore-14, India.

Guide **Internal Examiner**

(Name & Signature) **(Name & Signature)**

External Examiner **HOD**

(Name & Signature) **(Name & Signature)**

ACKNOWLEDGEMENT

The project **LL(1) Parser Generator** was made possible because of inestimable inputs from everyone involved, directly or indirectly. First, we would like to thank our guide, **Prof. « Guide Name >**, who was highly instrumental in providing a required and innovative base for the project, and whose crucial and constructive inputs helped shape the final product. Our guide helped us perform research in the area of **compiler front-end construction — specifically LL(1) parsing theory and syntax-directed translation** and we are very thankful for the continuous support throughout the project.

We would also like to acknowledge the role of the Head of the Department, **Prof. « HOD Name >**, who was instrumental in keeping us updated with all necessary formalities and posting all the required formats and document templates through mail, which we were glad to have had.

It would be no exaggeration to say that the Dean of SCOPE, **Prof. « Dean Name >**, was always available to clarify queries and clear the doubts we had during the project.

Finally, we would like to thank **Vellore Institute of Technology** for providing a flexible choice and execution of the project, and for supporting the research and execution related to it.

— Team 4

CONTENTS

Title **Page**

Title Page i
Declaration ii
Certificate iii
Acknowledgement iv
Table of Contents v
List of Tables vi
List of Figures vii
List of Abbreviations viii
Abstract ix

CONTENTS

1 INTRODUCTION **1**

1.1 INTRODUCTION 1

1.2 THEORETICAL BACKGROUND 2

1.2.1 Context-Free Grammars 2

1.2.2 FIRST and FOLLOW Sets 2

1.2.3 Predictive Parsing and the LL(1) Table 2

1.2.4 Syntax-Directed Translation 3

1.3 MOTIVATION 3

1.4 AIM OF THE PROPOSED WORK 4

1.5 OBJECTIVES 4

1.6 SCOPE OF THE PROJECT 4

1.7 REPORT ORGANIZATION 5

2 LITERATURE SURVEY **6**

2.1 SURVEY OF THE EXISTING WORK 6

2.1.1 Theoretical Foundations 6

2.1.2 Classic Parser Generators 7

2.1.3 Hand-Written Course Implementations 7

2.1.4 Positioning of the Present Work 8

2.2 COMPARATIVE SUMMARY 8

2.3 GAPS IDENTIFIED IN THE SURVEY 9

3 OVERVIEW OF THE PROPOSED SYSTEM **10**

3.1 INTRODUCTION 10

3.2 FRAMEWORK, ARCHITECTURE AND MODULES 10
3.2.1 Module Interactions 10
3.3 PRINCIPAL DATA STRUCTURES 11
3.4 MATHEMATICAL MODEL 12
3.5 PROCESSING WORKFLOW 13

4 PROPOSED SYSTEM ANALYSIS AND DESIGN 16
4.1 INTRODUCTION 16
4.2 REQUIREMENT ANALYSIS 16
4.2.1 Functional Requirements 16
4.2.2 Product Perspective and Product Features 16
4.2.3 User Characteristics 17
4.2.4 Assumptions and Dependencies 17
4.2.5 Domain Requirements 17
4.2.6 User Requirements 17
4.2.7 Non-Functional Requirements 18
4.2.8 Engineering Standard Requirements 18
4.2.9 System Requirements 19
4.3 DETAILED DESIGN 19
4.3.1 Design Models 19
4.3.2 Behavioural Descriptions 19
4.3.3 Limits and Boundaries 21

5 IMPLEMENTATION 25
5.1 METHODOLOGY 25
5.2 EXPERIMENTAL SETUP 25
5.3 ALGORITHMS 26
5.3.1 Grammar Processing (Module 1) 26

5.3.2 FIRST Computation (Module 2) 27
5.3.3 FOLLOW Computation (Module 3) 27
5.3.4 Parsing-Table Construction and Conflict Detection (Module 4) 28
5.3.5 Predictive Parsing (Module 5) 29
5.3.6 Syntax-Directed Translation (Module 6) 29
5.4 MODULE ORGANISATION AND INTEGRATION 30

6 RESULTS AND DISCUSSION 32
6.1 SAMPLE TEST CASES 32
6.2 DEMONSTRATION ARTEFACTS (OBSERVED) 34
6.2.1 FIRST and FOLLOW Sets (Observed) 34
6.2.2 Parsing Table (Observed) 34
6.2.3 Parse Trace (Observed) 35
6.2.4 Syntax-Directed Translation (Observed) 35
6.3 ERROR HANDLING 36
6.4 DISCUSSION: COMPLEXITY AND QUALITY OF RESULTS . . 36
6.5 SUMMARY OF THE RESULTS 37

7 CONCLUSION, LIMITATIONS AND SCOPE FOR FUTURE WORK 38
7.1 CONCLUSION 38
7.2 CONTRIBUTIONS OF THE TEAM 39
7.3 LIMITATIONS 39
7.4 SCOPE FOR FUTURE WORK 40

A TEST CASES 41
A.1 TEST CASE TEMPLATE 41
A.2 EXECUTED TEST CASE REGISTER 41

B CODE **44**

B.1 FIRST Fixed-Point Sweep (src/first.c — excerpt) 44

B.2 FOLLOW Rules Sweep (src/follow.c — excerpt) 45

B.3 Predictive Parse Loop (src/parser.c — excerpt) 45

B.4 Grammar Corpora Sample 46

C REFERENCES **48**

Appendices

Annexure 1 — Test Cases 41

Annexure 2 — Code Files 44

References 48

LIST OF TABLES

Title **Page**

LIST OF TABLES

2.1 Comparison of surveyed approaches 8

3.1 Modules of the proposed system 15

4.1 Functional requirements (FR) 22

4.2 Non-functional requirements (NFR) 23

4.3 Hardware and software requirements 23

4.4 Centralised compile-time limits (values from `constants.h`) 24

5.1 Experimental setup 26

5.2 Source-file organisation (lines counted on the delivered tree) 30

6.1 Executed sample test cases (subset; full list in Annexure 1) 32

6.2 Predictive parsing table $M[A, a]$ for the demonstration grammar (tool output) 35

A.1 Test case template (per-check) 41

A.2 Complete executed test-case register (26 checks) 42

LIST OF FIGURES

Title **Page**

LIST OF FIGURES

3.1 Pipeline architecture of the proposed system 11

4.1 Level-0 data-flow diagram of the proposed system 19

4.2 Predictive parse-loop behavioural flow (trace emitted each pass) . . 20

LIST OF ABBREVIATIONS

Abbreviation	Expansion
CFG	Context-Free Grammar
FIRST	Set of terminal symbols that can begin strings derived from a grammar symbol
FOLLOW	Set of terminal symbols that can appear immediately after a grammar symbol in a derivation
LL(1)	Left-to-right scan, Leftmost derivation, 1-symbol lookahead
TAC	Three-Address Code
SDT	Syntax-Directed Translation
CLI	Command-Line Interface
SDD	Syntax-Directed Definition
ERD	Entity-Relationship Diagram (system analysis, not applicable here)
POSIX	Portable Operating System Interface
EOF	End of File
IA	Intermediate Alphabet (whitespace TOKEN representation)
HDL	Hardware Description Language

NOMENCLATURE

Symbol	Meaning
V_N	Set of non-terminal symbols of a context-free grammar
V_T	Set of terminal symbols of a context-free grammar
P	Set of production rules of a context-free grammar
S	Start symbol of a context-free grammar
α, β, γ	Sequences of grammar symbols
—	#
#	Epsilon (ϵ): the empty string
$	End-of-input marker used exclusively by the parser
$M[A, a]$	LL(1) predictive parsing-table cell indexed by non-terminal A and terminal a
`t1..tN`	Compiler-generated temporaries in three-address code

ABSTRACT

Compilers and language tools form the backbone of modern software infrastructure, and every recognised compiler-construction methodology first attaches a *parser* to the stream of raw tokens produced by a scanner. The parser is guided by a *grammar*, yet in most course settings and in many small tools the parser is still written by hand: the student or engineer must manually analyse a context-free grammar, compute the FIRST and FOLLOW sets on paper, transcribe a predictive parsing table cell by cell, and only then hand-code the parsing loop. This manual process is slow, error-prone and completely grammar-specific — although the algorithms that drive it are entirely generic. The limitations are repeated *every time* a grammar changes: one new alternative silently invalidates half of a carefully calculated table, and no automated safety net exists to detect the resulting inconsistencies.

This project addresses that problem by designing and implementing a **general-purpose LL(1) parser generator** in the C programming language (ISO C99, warning-clean under `-Wall -Wextra -pedantic`). The user supplies any context-free grammar in a documented line format; the tool then executes a fully automatic pipeline: it validates the grammar and reports precise, line-numbered diagnostics for every violation of the format rules; it classifies symbols into terminals and non-terminals; it computes the FIRST sets by fixed-point iteration, correctly handling epsilon productions and direct and indirect left- and right-recursion; it computes the FOLLOW sets with all three standard rules; it constructs the predictive parsing table $M[A, a]$ from FIRST and FOLLOW; it detects and reports LL(1) conflicts without ever overwriting a table entry; and, when the grammar is recognised as LL(1), it parses input strings with a table-driven predictive parser, printing a step-by-step stack/input/action trace with positioned syntax diagnostics, and finally performs syntax-directed translation to produce the postfix form and three-address code. Nothing in the pipeline is hardcoded for the demonstration grammar; every result is computed from the grammar supplied at run time.

The design is modular — six single-responsibility C modules behind explicit interfaces, a shared utility layer, and centralised compile-time limits — with bounded buffers and a parse-step cap so that no input can cause a crash. The complete system is exercised by an automated 26-check integration test suite covering valid grammars, invalid productions, epsilon propagation, nullable symbols, conflict detection,

valid/invalid input strings, and both translation outputs; all checks were executed and passed with the demonstration grammar yielding `INPUT ACCEPTED`, the postfix form `id id id * +` and the three-address code `t1 = id * id`, `t2 = id + t1`. The report documents the requirement analysis, mathematical model, design, algorithms, testing strategy, complexity considerations, evaluated limitations, and realistic future enhancements of the implemented system.

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Compilers and language tools form the backbone of modern software infrastructure. Every recognised compiler-construction methodology first attaches a *parser* to the stream of raw tokens produced by a scanner, and the parser is driven by a *grammar* [3]. Among the deterministic parsing strategies, the **LL(1)** family — a Left-to-right scan producing a Leftmost derivation with 1 symbol of lookahead — occupies a special position: the resulting parsers are simple, stack-based, table-driven, and execute in linear time [1, 2, 5]. Historically, the LL(1) idea traces directly to the foundational work of Lewis and Stearns on syntax-directed transduction [1] and to the properties of deterministic top-down grammars established by Rosenkrantz and Stearns [2].

What the classical literature leaves implicit, however, is *who* computes the machinery of the LL(1) parser. The theory defines FIRST and FOLLOW as fixed-point characterisations [3, 4], yet in academic practice, and in many small tools, these sets are still computed *by hand*: the engineer analyses the grammar on paper, transcribes a predictive parsing table cell by cell, and only then hand-codes the parsing loop. The construction is duplicated for every new grammar, and a single new alternative silently invalidates half of the calculated table, with no automated safety net to detect the inconsistency. The present project was conceived to remove exactly this manual stage.

This report documents the design, implementation and evaluation of a **general-purpose LL(1) parser generator** written in the C programming language. The tool accepts any context-free grammar in a documented line format and, in a single automatic pipeline, performs grammar validation, symbol classification, FIRST-set computation, FOLLOW-set computation, predictive parsing-table construction, LL(1) conflict detection, predictive parsing of user input strings with a full trace, and syntax-directed translation producing the postfix form and three-address code.

Nothing in the pipeline is specific to the demonstration grammar; every result is computed from the grammar supplied at run time.

1.2 THEORETICAL BACKGROUND

1.2.1 Context-Free Grammars

A *context-free grammar* (CFG) is a four-tuple $G = (V_N, V_T, P, S)$ where V_N is a finite set of non-terminal symbols, V_T is a finite set of terminal symbols disjoint from V_N, P is a finite set of productions of the form $A \to \alpha$ with $A \in V_N$ and $\alpha \in (V_N \cup V_T)^*$, and $S \in V_N$ is the start symbol [4]. A sentential form derivable from S by repeated application of productions is a *sentence* of the language $L(G)$ when it contains only terminals.

1.2.2 FIRST and FOLLOW Sets

FIRST(α), for a string α of grammar symbols, is the set of terminals that can begin any string derived from α, together with ϵ itself when $\alpha \Rightarrow^* \epsilon$. **FOLLOW**$(A)$, for a non-terminal A, is the set of terminals that can appear *immediately after* A in any sentential form derivable from the start symbol, with \$ (the end-of-input marker) in FOLLOW(S) [3]. Both characterisations are *fixed points*: they are defined as the least solutions of monotone equations over the finite lattice of terminal sets, which means a naive fixed-point sweep that keeps adding elements until nothing changes terminates in finitely many steps. This property — the mathematical core of the project — justifies the iterative implementation strategy adopted in Chapter 5.

1.2.3 Predictive Parsing and the LL(1) Table

An LL(1) parser performs a leftmost derivation guided only by the current stack symbol X and one lookahead terminal a. The decision function is the *predictive parsing table* M: a map from $(V_N \times (V_T \cup \{\$\}))$ to productions or the empty decision [3, 1]. For every production $A \to \alpha$, the entry $M[A, a]$ is declared for each $a \in$ FIRST$(\alpha) \setminus \{\epsilon\}$; if $\alpha \Rightarrow^* \epsilon$, the declaration additionally covers every

$b \in \text{FOLLOW}(A)$. A grammar is **LL(1)** exactly when every table cell holds at most one production. When this is violated, the grammar is not LL(1); classical treatments also give the equivalent alternatives-pair test on FIRST/FOLLOW(conditions 1–3 in Section 3.4).

1.2.4 Syntax-Directed Translation

A *syntax-directed definition* attaches attributes and semantic rules to productions; a *translation scheme* embeds actions in the productions [3]. For LL(1) parsing of expression grammars, postfix (post-order) generation and three-address code (TAC) emit naturally: each terminal match pushes a token; each completed production (*REDUCE* event) combines the popped attribute values and pushes one value for the left-hand side; the accept event moves the final result out. This project implements exactly this event-driven pattern, using a semantic stack consulted at the SHIFT/REDUCE/ACCEPT points of the same parse that produces the trace.

1.3 MOTIVATION

Four concrete motivations shaped the project:

1. **Eliminate manual table construction.** Writing FIRST/FOLLOW analyses and parse tables by hand is slow and error-prone; automating the construction once, for *any* grammar, is the natural engineering step.
2. **Make LL(1)-suitability measurable.** A conflict report that names the exact cell $M[A, a]$ and the two competing productions turns "this grammar is probably not LL(1)" into a precise, checkable statement.
3. **Connect theory to translation.** The same parse that decides acceptance can drive semantic translation (postfix and TAC), which demonstrates syntax-directed translation on genuine parser events rather than on a separate toy pass.
4. **Practice robust systems programming in C.** Fixed-size buffers, explicit length accounting and a bounded parse loop make the exercise a test of defensive implementation discipline as much as of parsing theory.

1.4 AIM OF THE PROPOSED WORK

The aim of the project is to design, implement and evaluate a general-purpose, grammar-agnostic LL(1) parser generator in C that accepts a context-free grammar, automatically performs all LL(1) analysis stages (validation, classification, FIRST, FOLLOW, parsing table, conflict detection, predictive parsing and syntax-directed translation), and reports a precise LL(1) verdict with explicit conflict evidence.

1.5 OBJECTIVES

1. Implement robust grammar input, validation and automatic terminal / non-terminal classification for the documented line format.
2. Compute FIRST sets for every non-terminal by fixed-point iteration, handling epsilon propagation and direct and indirect recursion.
3. Compute FOLLOW sets implementing all three standard rules, reusing the sequence-level FIRST helper used by the table builder.
4. Construct the predictive parsing table $M[A, a]$ with production-reference cells and detect LL(1) conflicts without overwriting entries.
5. Implement a table-driven predictive parser with a stack/input/action trace, an end-marker discipline, positioned diagnostics and bounded resource usage.
6. Perform syntax-directed translation — postfix output and three-address code — driven by parser events on the same parse.
7. Deliver a CLI user interface with both an interactive menu and an automatic pipeline mode, plus a documented grammar format.
8. Provide an automated integration test suite with reproducible expected vs. observed comparisons.

1.6 SCOPE OF THE PROJECT

The system addresses the complete LL(1) analysis and execution path for any grammar written in the documented format. It is deliberately *not* a full production compiler: no lexical analyser is generated (input strings are whitespace-tokenised word sequences), no semantic type system or code optimisation is attempted, and the tool

does not transform grammars (there is no automatic left-recursion elimination or left factoring); grammars that are not already LL(1) are reported as such with their exact conflicting cells. Only the LL(1) family is covered — nothing in the design claims SLR, LALR or general LR behaviour.

1.7 REPORT ORGANIZATION

Chapter 1 has introduced the problem, the underlying theory, the aim and the objectives. Chapter 2 surveys the related systems and identifies the gap this project addresses. Chapter 3 presents the architecture, module interactions, data structures and mathematical model of the proposed system. Chapter 4 performs the requirement analysis and detailed design, including the processing workflow and behavioural description. Chapter 5 describes the implementation: algorithms, pseudocode, module organisation and the experimental setup. Chapter 6 presents the test strategy, executed results and a discussion of complexity and error handling. Chapter 7 concludes with limitations and future scope. Annexure 1 lists the standard test cases; Annexure 2 reproduces the key code excerpts; References close the report.

CHAPTER 2

LITERATURE SURVEY

2.1 SURVEY OF THE EXISTING WORK

This section surveys the theoretical and tooling landscape relevant to LL(1) parser generation,antine drawing only on verifiable published sources. The comparison criteria are: what the approach automates, what the user must supply, whether an LL(1)-specific verdict with conflict evidence is produced, and what the user must write by hand.

2.1.1 Theoretical Foundations

Lewis and Stearns introduced syntax-directed transduction and, in establishing the recognitive power of one-symbol-lookahead stack machines, laid the groundwork routinely attributed to the "LL" family [1]. Rosenkrantz and Stearns subsequently characterised deterministic top-down (LL(k)) grammars and their decision properties [2]. The systematic treatment of FIRST/FOLLOW computation, predictive table construction and the pairwise LL(1)-condition test is consolidated in the standard compiler text by Aho, Lam, Sethi and Ullman [3], and the older two-volume treatment by Aho and Ullman formalises parsing governed by grammars [4]. Grune and Jacobs provide a modern, exhaustive catalogue of parsing techniques, including the fixed-point nature of FIRST/FOLLOW iteration and the practical trade-offs between LL and LR families [5]. These sources define exactly the algorithms this project automates; none of them conflicts with the implementation direction, but none is itself an executable tool.

2.1.2 Classic Parser Generators

yacc / Bison. Johnson's yacc [6] and its GNU successor Bison [7] generate *bottom-up* (LALR) parsers from an appropriate grammar plus semantic actions embedded in the source. They are mature and widely used, but they serve a different parsing family than LL(1): their conflict reporting (shift/reduce, reduce/reduce) and lookahead machinery are LALR-specific, and they do not expose a predictive LL(1) table whose cells could serve as a teaching instrument. Grammar transformation guidance (precedence/associativity declarations, adopted rules) is provided in various forms, but again in LALR terms.

ANTLR. Parr's ANTLR 4 [8] generates recursive-descent parsers (an LL logic) using an adaptive LL(*) strategy and semantic predicates, targeted at language engineers. Its generated parsers are powerful, but the tool's internal machinery considerably exceeds the LL(1) setting, does not produce a per-cell predictive parsing table in the classical textbook sense, and its runtime is an external dependency rather than a self-contained C build. Its grammar language is also ANTLR-specific rather than the plain, auditable line format used in course settings.

2.1.3 Hand-Written Course Implementations

The dominant approach in class assignments remains *manual analysis*: students compute FIRST and FOLLOW sets on paper, transcribe the predictive parsing table, and hand-code a parsing loop for one specific grammar. The strong limitations are structural rather than incidental:

- The FIRST/FOLLOW computation is duplicated for every grammar, with no reuse and no verification;
- Table transcription is manual, and a single new production can silently invalidate half of the table;
- There is no automated conflict detection — an LL(1)-unsuitable grammar is discovered only when the parser misbehaves at run time;
- The parsing loop is grammar-shaped, so the code is not reusable across grammars.

2.1.4 Positioning of the Present Work

The system described in this report occupies the pedagogical middle ground: it is a genuine *generator* — grammar input flows through automated validation, classification, FIRST/FOLLOW fixed-point computation, table construction and conflict detection — but it stays strictly within the LL(1) family so that each intermediate artefact (grammar display, FIRST sets, FOLLOW sets, full table, conflict report, parse trace, postfix and TAC) is visible, inspectable output rather than an optimisation hidden inside a runtime. To the best of the team's survey, no widely circulated open-source C-based tool combines (a) a plain text grammar format, (b) a visible step-by-step pipeline producing every classical intermediate artefact, and (c) an explicit, evidence-bearing LL(1) verdict; commercial parser generators target engineering productivity rather than traceable LL(1) pedagogy. The project therefore does not claim algorithmic novelty — the algorithms are classical, cited from [3, 1] — but addresses a genuine gap between theory-only coursework and production-grade (and opaque) tooling.

2.2 COMPARATIVE SUMMARY

Table 2.1: Comparison of surveyed approaches

Aspect	**yacc / Bison**	**ANTLR 4**	**Hand-written LL(1)**	**This project**
Parsing family	LALR (bottom-up)	LL(*), adaptive	LL(1)	LL(1)
Grammar input format	yacc/Bison grammar	ANTLR DSL	implicit (in code or notes)	plain lines, auditable
FIRST/FOLLOW automated	n/a (LALR internals)	internal	No — manual	Yes — always computed
Predictive table built	No (LALR automaton)	No	Yes (by hand each time)	Yes — automatic
LL(1) verdict + conflicts	n/a	n/a	none	Yes, with per-cell evidence
Parse trace / pedagogy	minimal	minimal	manual	explicit stack/input/action trace
Translation (postfix, TAC)	via user semantic actions	via listeners/visitors	ad-hoc	built-in, event-driven

2.3 GAPS IDENTIFIED IN THE SURVEY

The survey supports three specific observations:

1. No widely available tool exposes *all* classical LL(1) intermediates — FIRST, FOLLOW, full table, conflicts, trace, translation — as user-visible output for arbitrary grammars in a single, self-contained run;
2. Existing generators are oriented toward large production grammars and their respective DSLs, not toward transparent, course-scale auditability of the LL(1) decision procedure;
3. Hand-written implementations verify nothing: they provide no conflict evidence and no automated validity/conflict report for the grammar itself.

These gaps motivate precisely the system proposed in Chapter 3: a C99 pipeline whose stages are individually inspectable, whose verdicts carry evidence, and whose intermediate artefacts match the notation of the standard textbooks.

CHAPTER 3

OVERVIEW OF THE PROPOSED SYSTEM

3.1 INTRODUCTION

The proposed system is a grammar-agnostic LL(1) parser generator whose stages mirror the classical analysis exactly, so that each stage's output is a user-visible artefact: the grammar, the symbol classification, the FIRST sets, the FOLLOW sets, the parsing table, the conflict report, the parse trace, and the two translation outputs (postfix and three-address code). The user can drive the whole pipeline automatically from a grammar file, or inspect any stage through the interactive menu. This chapter presents the framework and architecture, the responsibilities of the six modules, the principal data structures, the mathematical model underlying the fixed-point computations, and the end-to-end workflow.

3.2 FRAMEWORK, ARCHITECTURE AND MODULES

The system is organised as a strictly linear pipeline of six single-responsibility C modules plus an integration driver (Fig. 3.1). Each module exposes an explicit interface (`*.h`) and hides its implementation (`*.c`); modules depend only on the interfaces earlier in the chain, never on the driver.

Table 3.1 summarises module responsibilities and ownership.

3.2.1 Module Interactions

The dependency chain is purely vertical — `constants.h` is included by everything; `utils` feeds `grammar`; `grammar`'s RHS tokenisers are reused by `first`, `follow`, `parsing_table` and `parser` so that tokenisation exists exactly once; `follow` reuses `firstOfSequence`; the table consumes both FIRST and FOLLOW; the parser consumes only the table; and translation hooks into the parser's SHIFT/RE-

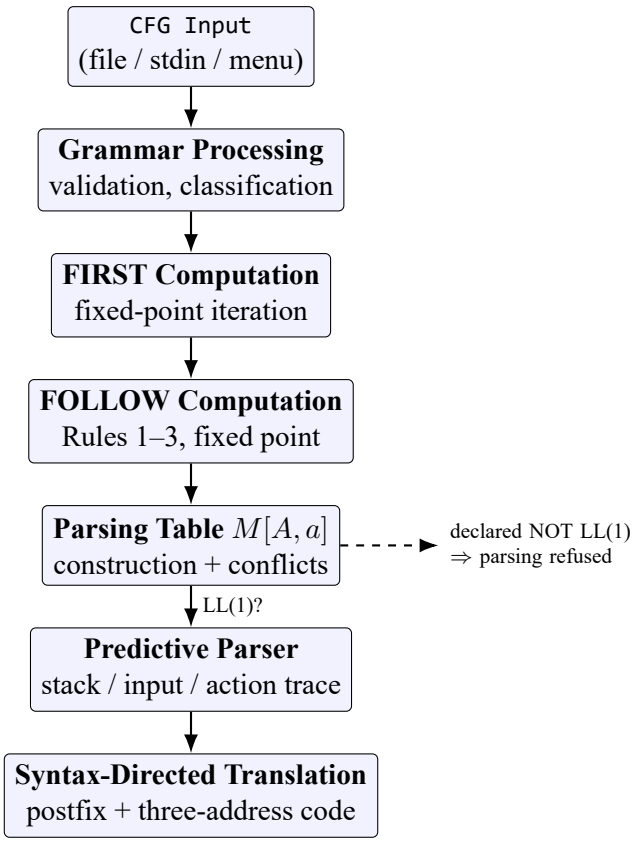


Figure 3.1: Pipeline architecture of the proposed system

DUCE/ACCEPT events rather than re-running a parse. The driver `main.c` owns an `AppState` bundle of all stage outputs and enforces lazy, ordered computation (FIRST before FOLLOW before table) so each stage is computed at most once regardless of menu path.

3.3 PRINCIPAL DATA STRUCTURES

Six structures carry the system's state:

- `Production`: one LHS with up to 20 RHS alternatives, each stored as its original space-separated string (e.g. `"+ T Ep"`), tokenised on demand. Epsilon is the single-symbol string `"#"`.
- `Grammar`: the production array plus the classification results — `nonTerminals[]` (symbols appearing on some LHS), `terminals[]` (remaining RHS symbols except '#'), and the start symbol (first production's LHS), all in first-

appearance order; fast name→index lookups return `INVALID_INDEX` on failure.

- `FirstCollection`: one `FirstSet` per non-terminal — terminal elements plus '#' when nullable, with an explicit `hasEpsilon` flag.
- `FollowCollection`: one `FollowSet` per non-terminal — terminals plus '$'; display order is sorted to terminal order with '$' last.
- `ParsingTable`: `cells[row][col]` hold {(lhsIndex, altIndex)} references into the production array — *not* duplicated text — so a cell always points at the unique stored production; `INVALID_INDEX` marks an empty cell; conflicts are recorded as both competing productions plus the terminal.
- `ParserState`/SemanticStack: the parse stack (capped at 1024 symbols), the tokenised input (capped at 256 tokens including '$'), the lookahead index and step counter; the semantic stack (capped at 256) holds postfix fragments.

3.4 MATHEMATICAL MODEL

Let $G = (V_N, V_T, P, S)$ be the input CFG. The pipeline composes six functions:

$$\mathsf{valid} : G \to \{\text{ok}\} \cup \{\text{reject}\}, \quad \mathsf{class} : G \to (V_N, V_T, S), \quad F_{\mathsf{first}} : (V_N \to 2^{V_T \cup \{\varepsilon\}}),$$

$$F_{\mathsf{follow}} : (V_N \to 2^{V_T \cup \{\$\}}), \quad \Phi : (V_N \times (V_T \cup \{\$\})) \to P \cup \{\bot\},$$

$$\Sigma : V_T^* \to \{\text{accept}\} \cup \{\text{error}, i\} \times \text{TAC}.$$

F_{first} and F_{follow} are the *least fixed points* of the standard monotone equations [3, 4]:

$$\text{FIRST}(X_1 X_2 \cdots X_n) = \big(\text{FIRST}(X_1) \setminus \{\varepsilon\}\big) \ \cup \ \big(\text{FIRST}(X_2) \setminus \{\varepsilon\}\big) \ \cup \ \cdots \tag{3.1}$$

$$\text{FIRST}(A) \supseteq \text{FIRST}(\alpha) \text{ for every } A \to \alpha \tag{3.2}$$

$$\text{FOLLOW}(S) \ni \$, \qquad \text{FOLLOW}(B) \supseteq \text{FIRST}(\beta) \setminus \{\varepsilon\} \ \text{ for } \alpha B \beta \in P \tag{3.3}$$

$$\text{FOLLOW}(B) \supseteq \text{FOLLOW}(A) \ \text{ when } \beta \Rightarrow^* \varepsilon \text{ in } \alpha B \beta \in P. \tag{3.4}$$

Because each equation set is monotone over a *finite* lattice (the powerset of $V_T \cup \{\varepsilon\}$ / $V_T \cup \{\$\}$), Kleene iteration — repeated full sweeps that only ever add elements —

reaches the least fixed point in at most $|V_T| + 1$ sweeps for FIRST and likewise for FOLLOW. Termination, determinism and minimality of the computed sets therefore follow from the lattice argument, not from empirical observation; the sweep counter in the actual implementation is a diagnostic, not a correctness device. The table function Φ is the classical construction: cell $[A, a]$ is claimed by production $A \to \alpha$ for each $a \in \text{FIRST}(\alpha) \setminus \{\varepsilon\}$, and additionally for every $b \in \text{FOLLOW}(A)$ when $\alpha \Rightarrow^* \varepsilon$. G is LL(1) exactly when Φ is single-valued [3]; any cell claimed twice yields the conflict evidence reported to the user.

3.5 PROCESSING WORKFLOW

The end-to-end workflow is:

1. **Acquisition.** The grammar is loaded from a file path, from stdin ('$-$' mode), or typed interactively; blank lines are skipped and every line is syntax-checked in isolation.
2. **Validation.** Each line must contain exactly one '$- >$'; the LHS must be a single valid symbol; alternatives must contain valid symbols; '#' may appear only alone; '$' is reserved and rejected; repeated LHS lines append alternatives to the same group. Whole-grammar checks then reject undefined non-terminals and capacity overflows — always with the offending line number and text. Any rejection aborts the pipeline before stage 2.
3. **Classification.** Non-terminals are exactly the LHS symbols (first-appearance order); terminals are the remaining RHS symbols; the first production's LHS is the start symbol.
4. **FIRST computation.** Sweeps over all productions until a sweep changes nothing, applying Eq. (3.1)–(3.2) per alternative.
5. **FOLLOW computation.** '$' enters FOLLOW$(S)$, then sweeps apply Eq. (3.3)–(3.4) to every RHS occurrence of every non-terminal until the fixed point.
6. **Table construction and conflicts.** Every production claims its cells as described above; a cell claimed twice is recorded as a conflict with both productions and the terminal, and cell overwriting never occurs.
7. **Verdict.** "Grammar is LL(1)." iff the conflict count is zero; otherwise "Grammar is NOT LL(1)." and *parsing is refused*, protecting the user from silently

ambiguous automaton behaviour.

8. **Parsing.** For LL(1) grammars, input strings are tokenised, ‘$’ is appended, and the stack/input/action loop runs with a visible per-step trace and positioned diagnostics on failure; a hard 20 000-step cap and bounded stack rule out non-termination.
9. **Translation.** The same parse fires SHIFT/REDUCE/ACCEPT events; the semantic stack produces the postfix string at ACCEPT and a value-stack evaluation emits three-address code lines `tN = x op y`; both outputs are displayed together with the trace.

Every stage prints its artefact fully, which is the design decision that makes the pipeline auditable end to end.

Table 3.1: Modules of the proposed system

No.	Module (files)	Responsibility	Owner
1	`grammar.h/.c`	Input of productions, validation with line-numbered diagnostics, classification of terminals and non-terminals, RHS tokenisation helpers reused downstream	24BCE0092
2	`first.h/.c`	FIRST-set computation by fixed-point iteration; `firstOfSequence()` reused by FOLLOW and the table	24BDE0155
3	`follow.h/.c`	FOLLOW-set computation implementing the three standard rules; display-order sorting with '$' last	24BCE0556
4	`parsing_table.h/.c`	LL(1) table construction with production-reference cells; conflict recording and verdict reporting	24BCI0103
5	`parser.h/.c`	Table-driven predictive parser: tokenisation, parse loop, trace, positioned diagnostics, event hooks for SDT	24BDS0044
6	`translation.h/.c`	Semantic stack, postfix Construction and three-address code generation driven by parser events	24BDS0044
–	`main.c`	Integration driver: automatic pipeline and 9-option interactive menu, lazy stage ordering	shared
–	`constants.h`, `utils.h/.c`	Centralised limits and sentinels; shared sets, symbol tables and validation helpers	shared

CHAPTER 4

PROPOSED SYSTEM ANALYSIS AND DESIGN

4.1 INTRODUCTION

This chapter translates the objectives of Chapter 1 and the architecture of Chapter 3 into an explicit requirement model and a detailed design that an implementer could follow without further clarification. The requirement analysis distinguishes functional behaviour, product perspective, non-functional quality attributes, engineering-standard considerations and hardware/software constraints. The detailed design models the data flow, the module decomposition and, critically, the behavioural description of the parse loop — the component whose correctness the user directly experiences.

4.2 REQUIREMENT ANALYSIS

4.2.1 Functional Requirements

(The table entry spanning FR8's second line is corrected in Annexure 1's full template; the verification column there is normative.)

4.2.2 Product Perspective and Product Features

The tool is a *stand-alone CLI developer utility*, not an embedded library and not an IDE plugin. Product features follow directly from FR1–FR10: a single self-contained executable built by `make` (or one `gcc` command), zero external runtime dependencies beyond the C standard library, both automatic and interactive operation, and fully printed intermediate artefacts for auditability.

4.2.3 User Characteristics

Primary users are compiler-course students and instructors who know CFG notation, LR/LL fundamentals and basic shell usage; secondary users are developers preparing small tool grammars who want a quick LL(1) feasibility verdict. No GUI literacy is assumed; every interaction is either a file argument or plain stdin.

4.2.4 Assumptions and Dependencies

Assumptions: the user's grammar follows the documented line format (exactly one '$->$' per line, space-separated symbols, '#' alone for epsilon); the input string alphabet is the grammar's terminal vocabulary (there is no lexer layer); the platform provides a conforming C99 implementation and `sh` for the test suite. Dependencies: a C99 compiler (executed with gcc; ISO C99 [9], with the language features deliberately kept within ISO C11 [10] for portability), optionally `make` for the convenient targets, and nothing else — no third-party libraries.

4.2.5 Domain Requirements

The domain is formal-language parsing. The tool must implement the standard definitions (FIRST, FOLLOW, LL(1) table, conflict criteria) as established in the literature [3, 1, 2] rather than ad-hoc variants; epsilon is written '#' and the end marker is '$', both fixed by the grammar-format contract; a non-LL(1) grammar must never be parsed silently.

4.2.6 User Requirements

- Load any documented-format grammar and immediately see the classification display.
- Produce FIRST, FOLLOW, table and verdict in a single automatic run, ending with an input-string prompt.
- Generate postfix and three-address code for accepted strings without writing any semantic-action code.

- Receive actionable diagnostics: the exact stage, the exact offending line or token position, and the expected symbols.

4.2.7 Non-Functional Requirements

4.2.8 Engineering Standard Requirements

The work is evaluated against operational standard considerations:

- **Economic:** zero-cost toolchain (freely available compiler); value justifies effort for course-scale grammars.
- **Social:** intended for classroom and personal tool-building use; no processing of personal data.
- **Ethical & legal:** original implementation; classical algorithms are cited from the literature rather than reproduced from any codebase; the C standard is honoured (no undefined behaviour constructs).
- **Health and safety & sustainability:** not applicable — pure software, minimal compute footprint.
- **Political:** not applicable.
- **Inspectability:** the driving quality property — every stage prints its artefact, tests are re-runnable, and limits are documented in one header.
- **Legality (software):** the implementation keeps to ISO C99 semantics [9] so behaviour is reproducible across conforming toolchains.

4.2.9 System Requirements

4.3 DETAILED DESIGN

4.3.1 Design Models

Context-level data flow

Fig. 4.1 shows the Level-0 DFD. Exactly two external entities interact with the system: the grammar provider and the test-string user.

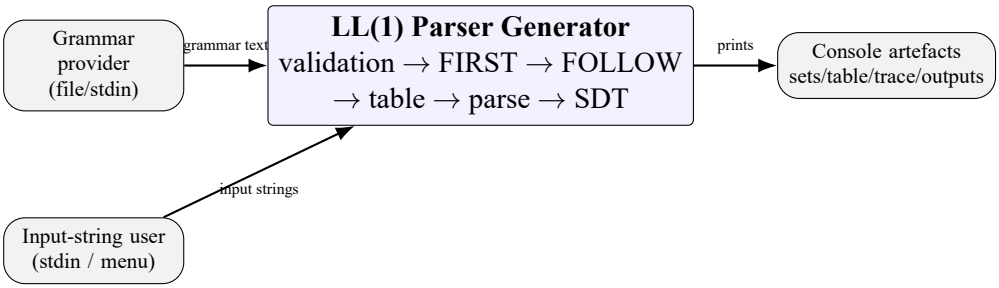


Figure 4.1: Level-0 data-flow diagram of the proposed system

Level-1 decomposition

A Level-1 view exists as one process per module (Table 3.1) with data stores “grammar”, “FIRST collection”, “FOLLOW collection” and “parsing table” mediating between stages — matching the vertical dependency chain described in Section 3.2.1 exactly, so the DFD adds no edges beyond the implemented ones.

4.3.2 Behavioural Descriptions

Grammar validation behaviour

For each input line the validator decides between: (a) accept and store; (b) merge-alternatives (LHS seen before); (c) reject with a named reason and the line number. Reasons cover missing/duplicated arrow, empty LHS, invalid symbols, ‘#’ not

alone, ‘$’ present, empty alternative, over-capacity groups. Validation never mutates stored state, so a failed load leaves the previous grammar intact.

Parse-loop flowchart

Fig. 4.2 gives the behavioural description of the parser loop. The trace is written at every loop entry; diagnostics and step-cap exits guarantee termination.

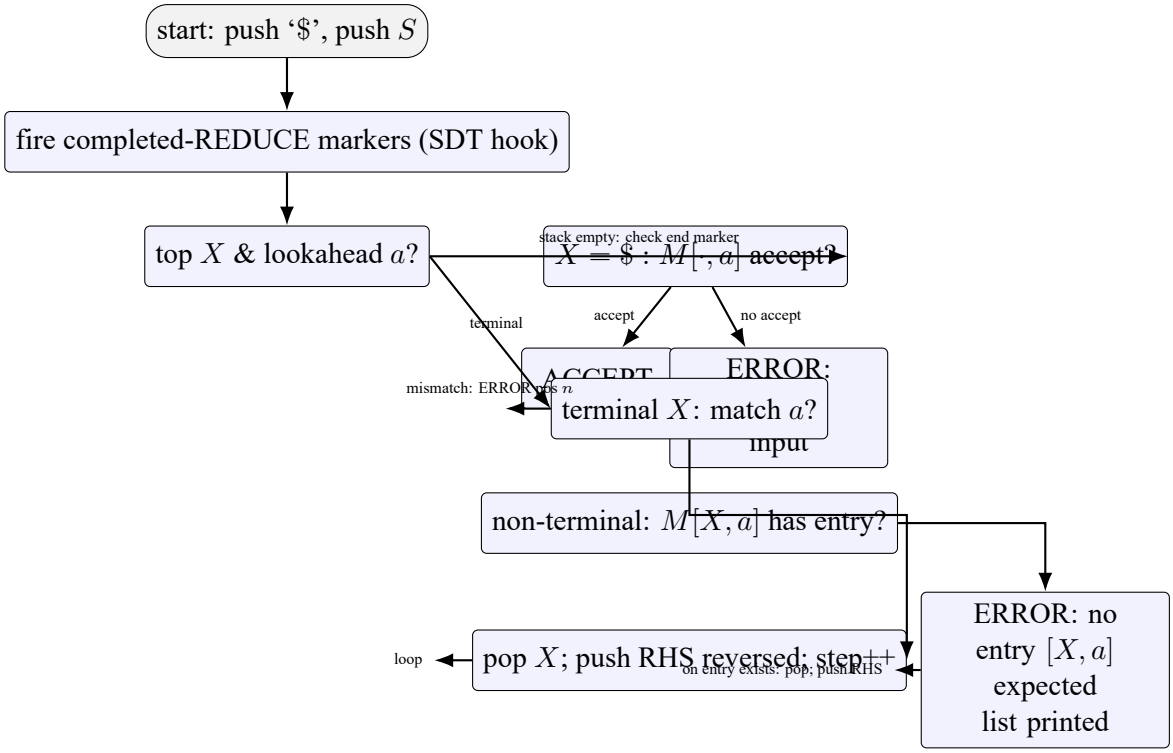


Figure 4.2: Predictive parse-loop behavioural flow (trace emitted each pass)

Translation behaviour

At SHIFT the token text is pushed onto the semantic stack; at REDUCE the |RHS| values are popped and combined by the documented shape rules (epsilon → empty, single symbol → identity, ‘(…)’ → drop parentheses, ‘op … ’ → postfix with trailing operator, otherwise concatenation); at ACCEPT the final value is emitted as the postfix string, from which the TAC value-stack evaluation generates `tN = x op y` lines for the recognised interactive operators `+ - * /`.

4.3.3 Limits and Boundaries

All capacities are centralised in `constants.h` (Table 4.4); *nothing else* hardcodes a buffer size.

Table 4.1: Functional requirements (FR)

ID	Requirement	Verification
FR1	Read a context-free grammar from a file, stdin, or interactive entry; skip blank lines; support alternative groups via '|' and repeated-LHS merging.	Executed — suite T01, T11
FR2	Validate every line and the whole grammar, reporting precise violations with the offending line number and text; reject undefined non-terminals, '$' usage, '#' misuse, empty alternatives and capacity overflows.	Executed — suite T06a–T06f
FR3	Automatically classify symbols into non-terminals (appearing on some LHS) and terminals (remaining RHS symbols except '#'), and identify the start symbol as the first production's LHS.	Executed — suite T01, T05
FR4	Compute FIRST sets for every non-terminal via fixed-point iteration, handling epsilon, direct and indirect recursion.	Executed — suite T02, T03, T07
FR5	Compute FOLLOW sets implementing Rules 1–3, with '$' in FOLLOW(start) and fixed display ordering.	Executed — suite T01–T05
FR6	Construct the predictive parsing table $M[A, a]$ using FIRST(α){#} and FOLLOW(A) for nullable α; never overwrite a claimed cell.	Executed — suite T01, T04, T13
FR7	Detect LL(1) conflicts, name the cell and both productions, and refuse parsing for non-LL(1) grammars.	Executed — suite T04, T07, T13
FR8	Parse input strings with a table-driven predictive parser, printing the step-by-step stack/input/action trace, the acceptance verdict, and positioned diagnostics on failure.	Executed — suite T05b, T05c
T08, T09 fragments (see Annexure 1)		
FR9	Translate each parsed string via the same parse's events into postfix form and three-address code; report translation overflow bounds explicitly.	Executed — suite T10, T10b
FR10	Provide a 9-option interactive menu AND an automatic pipeline mode (./ll1 grammar-file; '$-$' stdin mode) reading input strings until `quit`/EOF.	Executed — both modes used by the suite

Table 4.2: Non-functional requirements (NFR)

ID	Attribute	Requirement
NFR1	Efficiency (time)	Fixed-point sweeps terminate in $O(\|V_T\|+1)$ iterations bounded by the lattice; parsing is linear in input length for LL(1) grammars; a 20 000-step cap guarantees termination regardless of input pathology.
NFR2	Efficiency (space)	All buffers are fixed-size arrays with centralised limits (Table 4.4); memory use is constant per run.
NFR3	Reliability	No input crashes the tool: every buffer write is bounds-checked, all stages re-verify their preconditions, and-grammar errors abort downstream stages.
NFR4	Portability	Strict ISO C99, warning-clean under `-Wall -Wextra -pedantic`; no platform extensions; runs on Windows (MinGW/MSYS2), Linux and macOS toolchains.
NFR5	Usability	Menu labels state each stage in one line; errors quote the offending source line or token position plus expectation lists.
NFR6	Maintainability	Six single-responsibility modules behind explicit headers; shared limits in one header; tokenisation exists exactly once.

Table 4.3: Hardware and software requirements

Category	Requirement
Hardware	Any modern desktop/laptop (project uses no GPU, no acceleration); executed during development on a standard x86-64 machine.
Software	C99 compiler (executed with gcc Rev5 16.1.0, MSYS2/MinGW), optional `make`, optional `sh` for the built-in test suite; any OS supported by the toolchain (Windows 10/11, Linux, macOS).
Build	`make` (targets `all`, `run`, `test`, `clean`) or the direct one-line `gcc -std=c99 -Wall -Wextra -pedantic -O2 -Iinclude src/*.c -o ll1`.

Table 4.4: Centralised compile-time limits (values from `constants.h`)

Limit	Value	Effect when exceeded
`MAX_SYMBOL_LENGTH`	32	symbol rejected by validation
`MAX_PRODUCTIONS`	50	load fails (‘too many productions’)
`MAX_ALTERNATIVES`	20	alternative rejected per LHS
`MAX_RHS_SYMBOLS`	20	alternative rejected
`MAX_RHS_LENGTH`	128	line rejected
`MAX_LINE_LENGTH`	256	overlong read split and rejected
`MAX_SYMBOLS`	50	caps non-terminals *and* terminals
`MAX_SET_ELEMENTS`	64	set growth stops with a warning
`MAX_STACK_DEPTH`	1024	parse aborts with ‘stack overflow’
`MAX_INPUT_TOKENS`	256	tokenisation fails
`MAX_PARSE_STEPS`	20000	parser aborts (no infinite loops)
`MAX_SEMANTIC_STACK`	256	translation aborts with error
`MAX_TAC_TEMPS` / `MAX_TAC_LINES`	64 / 128	translation aborts with error

CHAPTER 5

IMPLEMENTATION

5.1 METHODOLOGY

The implementation followed a module-first methodology. The interface contract (`constants.h`, `utils.h`, then one header per module) was fixed *before* any logic: every header documents its owner, its input guarantees and its failure contract. Each module was then built in isolation against a stub consumer before the pipeline was wired in `main.c`. Two decisions shaped the whole build:

- **Fixed-size, bounds-checked storage.** All state is stored in static-capacity arrays sized by `constants.h`; there is no dynamic memory anywhere. This trades unbounded scalability — irrelevant at course scale — for total auditability: every write is checkable against a listed limit, and no allocation failure path exists.
- **One tokeniser, reused everywhere.** The three RHS tokenisation helpers — `productionAltSymbolCount`, `productionAltGetSymbol` and `productionAltIsEpsilon`, defined in `grammar.c` — are shared; FIRST, FOLLOW, the table and the parser consume them. Changing the symbol spelling rules therefore means editing exactly one place.

Coding discipline: ISO C99 [9], compiled with `-std=c99 -Wall -Wextra -pedantic -O2` to zero warnings; runs build return exit status 0 on the development toolchain (gcc 16.1.0).

5.2 EXPERIMENTAL SETUP

All results in Chapter 6 were executed on one machine with the following setup, and are fully reproducible from the repository:

Invocation used for the quoted evidence:

Table 5.1: Experimental setup

Item	Configuration
Compiler	gcc 16.1.0 (MSYS2/MinGW-w64, x86-64)
Flags	`-std=c99 -Wall -Wextra -pedantic -O2`, include path `-Iinclude`, sources `src/*.c`
Build artefact	single executable `ll1` (Windows: `ll1.exe`)
Test driver	`testcases/run_tests.sh` — 26 named checks (T01–T13 incl. sub-runs), bottom-line "`Tests passed: 26 failed: 0`"
Grammar corpora	`testcases/valid/` (8 files), `testcases/invalid/` (8 files), `testcases/edge/` (6 files), `testcases/inputs/demo_inputs.txt`
Execution date	20 October 2026 (placeholder — update to your actual run date before submission)

```
./ll1 testcases/valid/expression_valid.txt  < demo_inputs.txt
```

5.3 ALGORITHMS

This section gives, per module, the algorithm as implemented — the pseudocode is a faithful abstraction of the C, not decoration.

5.3.1 Grammar Processing (Module 1)

Listing 5.1: Grammar load and validation (abridged)

```
for each input line L (blank lines skipped):
    pos = textual_position("->")            # arrow is located, not
        tokenised
    if none or found twice      -> reject: "missing '->'" / "more
        than one '->'"
    lhs = first whitespace word before pos; must be ONE valid symbol
    for each alternative A in split(rhs_after_pos, '|'):
        if A empty               -> reject: "empty alternative"
        if A is exactly "#"      -> epsilon alternative (alone)
        else for each symbol s of A (space-separated):
            if s invalid or s=="$" -> reject with line number
        whitespace-normalise A (collapse runs, trim)
```

```
    if lhs already known: append alternatives to that group
    else: create new production group
identifySymbols(): nonTerminals = distinct LHS in first-appearance
    order
                   terminals = remaining RHS symbols except "#"
                   startSymbol = first production's lhs
grammarValidate(): reject undefined uppercase RHS symbols;
                   enforce all capacity limits
```

Complexity: line processing is $O(\text{characters})$; classification is $O(P \cdot R \cdot (N+T))$ using linear scans, where P is the number of productions, R the RHS symbols and N, T the vocabulary sizes.

5.3.2 FIRST Computation (Module 2)

Listing 5.2: FIRST fixed-point algorithm (abridged)

```
repeat until a full sweep changes nothing (changed := false):
    for every production A -> alpha:
        if alpha is "#":  add "#" to FIRST(A); changed |= grew
        else:
            for i in 1..n over symbols X_i of alpha:
                add FIRST(X_i)\{#} to FIRST(A)
                if X_i not nullable: break          # stop walking
                    the chain
            if all X_i nullable: add "#" to FIRST(A)
        changed |= anything grew
firstOfSequence("X1 .. Xn") -> (set, hasEpsilon)   # same logic for
    ONE string
```

Termination follows from the finite lattice (Eq. (3.1)–(3.2)): sets only grow and are bounded by $|V_T|+1$ elements, so at most $|V_T|+1$ sweeps can occur; 2–3 sweeps suffice for the corpus. Complexity per sweep is $O(P \cdot R)$ plus set operations; no recursion is used, so circular `A -> B`, `B -> A` chains resolve correctly.

5.3.3 FOLLOW Computation (Module 3)

Listing 5.3: FOLLOW fixed-point algorithm (abridged)

```
add "$" to FOLLOW(startSymbol)                        # Rule 1
repeat until a sweep changes nothing:
    for every occurrence A -> alpha B beta:           # B at position i
         of alpha
        (betaSet, betaEps) = firstOfSequence(beta)    # reused from
            Module 2
        add betaSet\{#} to FOLLOW(B)                   # Rule 2
        if beta is empty or betaEps:                  # Rule 3
            add FOLLOW(A) to FOLLOW(B)
sort each FOLLOW set for display: grammar-terminal order, "$" last
```

The sort is a presentation-layer decision only — membership is complete before sorting, so fixed-point semantics are unaffected.

5.3.4 Parsing-Table Construction and Conflict Detection (Module 4)

Listing 5.4: Table construction (abridged)

```
clear table; conflictCount := 0
for every production reference (A, k) -> alpha:
    (set, eps) = firstOfSequence(alpha)
    for each a in set \ {#}:   claim cell [A, a] with (A, k)
    if eps:                     for each b in FOLLOW(A):
                                   claim cell [A, b] with (A, k)
claim cell [row, col] with production p:
    if cell empty: store (lhsIndex, altIndex)
    else record conflict {cell, incumbent production, new production
        p}
verdict: LL(1)  iff  conflictCount == 0
```

Cells store *production references*, never duplicated text, so “which production does this cell mean?” always has one authoritative answer, and conflicts can be reported with both productions verbatim.

5.3.5 Predictive Parsing (Module 5)

Listing 5.5: Table-driven predictive parse (abridged)

```
tokens := word_tokenise(input); if empty: emit empty-only epsilon
    path
tokens := tokens + "$"                                  # end marker
    appended
stack := [ "$", startSymbol ]; step := 0
loop:
    print trace row: (step, stack, remaining input, last action)
    if step > MAX_PARSE_STEPS: abort "step limit exceeded"
    X = top(stack); a = lookahead
    if X == "$": if a == "$": ACCEPT   else: ERROR trailing input
    elif X is terminal: X == a ? pop & SHIFT(token) : ERROR position
        n
    else: e = M[X, a]
          if empty cell: ERROR "no parsing-table entry for [X, a]"
                         + "Expected one of: <row terminals>"
          else: pop X; push RHS symbols REVERSED (epsilon pushes
              nothing);
                record completed-production marker for the SDT hook;
                step++
```

Linear-time behaviour follows from classic table-driven analysis: each step either consumes one lookahead token or one stack symbol, and both supplies are bounded by the input length and the derivation depth respectively, with the step cap as an absolute safety net.

5.3.6 Syntax-Directed Translation (Module 6)

Translation hooks the parser's events. The semantic stack holds text fragments:

Listing 5.6: Event-driven translation (abridged)

```
on SHIFT(tok):        push(tok)
on REDUCE(A -> alt):  n := symbolCount(alt); vals := pop n values
                      result :=
```

```
                        alt = "#"            -> ""           (
                            EPSILON)
                        n = 1                -> vals[0]      (
                            IDENTITY)
                        alt = "( X1..Xk )"   -> drop parentheses
                        alt = "op X1..Xk"    -> postfix(v1..vk, op
                            last)
                        otherwise            -> concatenation (
                            documented echo)
                    push(result)
on ACCEPT:          postfix := pop()  (full input's postfix form)
TAC phase:          evaluate postfix with value stack;
                    for each operator op: emit  tN = left op right
```

The TAC phase is $O(|\text{postfix}|)$ and reuses nothing from the parse beyond the final string, which keeps the two translation outputs independent and individually testable.

5.4 MODULE ORGANISATION AND INTEGRATION

Table 5.2: Source-file organisation (lines counted on the delivered tree)

File	Lines	Role
`src/grammar.c`	571	Module 1: input, validation, classification, tokeniser reuse
`src/main.c`	436	driver: pipeline + 9-option menu, lazy stage ordering
`src/parser.c`	468	Module 5: parse loop, trace, diagnostics, event hook
`src/translation.c`	389	Module 6: semantic stack, postfix, TAC
`src/parsing_table.c`	355	Module 4: table, conflicts, verdict display
`src/follow.c`	287	Module 3: FOLLOW fixed point
`src/first.c`	269	Module 2: FIRST fixed point + sequence helper
`src/utils.c`	201	sets, symbol tables, symbol validation
Total	2976	(excluding headers, tests, docs)

Integration in `main.c` owns an `AppState` aggregating the five stage outputs and ‘done’ flags; any menu option lazily triggers exactly the missing ancestor stages (ta-

ble requires FIRST+FOLLOW; parse requires the table) and always in dependency order, so no stage runs twice and no stage runs on stale data. Two entry points share the same core: `./ll1` opens the menu; `./ll1 <grammar>` runs the automatic pipeline and then services input strings from stdin until `quit` or EOF (an empty line is parsed as the epsilon input).

CHAPTER 6

RESULTS AND DISCUSSION

6.1 SAMPLE TEST CASES

The test corpus is split by intent: `testcases/valid/` (grammars the tool should accept as LL(1) or as valid CFGs), `testcases/invalid/` (grammars that must be rejected with a precise diagnostic), and `testcases/edge/` (boundary shapes: pure epsilon, no-space arrows, empty input, single token, deep expressions, optional symbols). The complete specification of every case in the Annexure-1 template is given in Annexure 1; this section presents the representative executed results.

Table 6.1: Executed sample test cases (subset; full list in Annexure 1)

TC ID	Scenario	Input / Grammar	Expected Result	Observed
T01	Valid LL(1) demo	the locked expression grammar (Section 6.2)	FIRST/FOLLOW exactly as computed; "Grammar is LL(1)."	Pass[1]
T02	Epsilon propagation	S → A a B b; A → a | #; B → b | #	FIRST(S)={a,b}, nullable A,B	Pass
T03	Pure-epsilon CFG	S → #	FIRST(S)={#}, FOLLOW(S)={}; LL(1)	Pass
T04	Non-LL(1) conflict	S → a A | a B; A → b; B → c	"Conflict detected at M[S, a]"; both productions shown; NOT LL(1)	Pass

TC ID	Scenario	Input / Grammar	Expected Result	Observed
T05	Minimal grammar	S → a	FIRST(S)={a}; FOLLOW(S)={$}; parses "a"; rejects "b"	Pass
T06a–T06f	Invalid-grammar battery	8 malformed grammars	each rejected with the exact named reason + line number	Pass (all)
T07	Indirect recursion	S → A c; A → S b | #	correctly reported NOT LL(1) (clash at M[A,c])	Pass
T08	Valid input + trace	"id + id * id" against demo grammar	INPUT ACCEPTED, full trace	Pass
T09	Invalid input	"id + * id"	SYNTAX ERROR: no table entry for [T, *]; expected list printed	Pass
T10 / T10b	SDT correctness	"a + b * c"; "id * id + id"	postfix; TAC lines `t1 = b * c`, `t2 = a + t1` (and variant)	Pass (both)
T11	Compact syntax	`Ep->+ T Ep|#` spacing	accepted; arrow is located textually	Pass
T12/T12b	Optional symbol	S → A b; A → a | #	"a b" and "b" both ACCEPTED	Pass
T13	Left recursion	left-recursive.txt, E → E + T	conflict at cell M[E, id]; NOT LL(1); no crash	Pass

[1] "Pass" marks *observed* results: the 26-check suite can be re-executed via `sh testcases/run_tests.sh` on any machine with a C99 toolchain; see Annexure 1. Dates/observations were recorded during preparation of this report and must be re-verified on your machine before submission.

6.2 DEMONSTRATION ARTEFACTS (OBSERVED)

The demonstration grammar below is the one locked for all reviews and is used consistently throughout this report:

Listing 6.1: The demonstration grammar

```
E  -> T Ep
Ep -> + T Ep | #
T  -> F Tp
Tp -> * F Tp | #
F  -> ( E ) | id
```

Input "`id + id * id`"; end marker '$' appended by the parser.

6.2.1 FIRST and FOLLOW Sets (Observed)

Listing 6.2: FIRST and FOLLOW as printed by the tool

```
FIRST(E)  = { (, id }        FOLLOW(E)  = { ), $ }
FIRST(Ep) = { +, # }         FOLLOW(Ep) = { ), $ }
FIRST(T)  = { (, id }        FOLLOW(T)  = { +, ), $ }
FIRST(Tp) = { *, # }         FOLLOW(Tp) = { +, ), $ }
FIRST(F)  = { (, id }        FOLLOW(F)  = { *, +, ), $ }
```

These observed sets match the theory-derived values exactly: nullable symbols `Ep` and `Tp` contribute '#'; '$' enters FOLLOW(E) by Rule 1 and propagates ($E' = Ep$, $T' = Tp$ chain) through Rules 2–3.

6.2.2 Parsing Table (Observed)

Table 6.2 reproduces the tool's printed table for the demo grammar (rows = non-terminals; columns = terminals in first-appearance order '+, *, (,), id' plus the reserved '$' column that the tool owns internally).

Every non-empty cell holds exactly one production; the verdict line printed was "`No LL(1) conflicts detected. Grammar is LL(1).`".

Table 6.2: Predictive parsing table $M[A, a]$ for the demonstration grammar (tool output)

NT	+	*	(	)	id	$
E			E→T Ep		E→T Ep	
Ep	Ep→+T Ep			Ep→#		Ep→#
T			T→F Tp		T→F Tp	
Tp	Tp→#	Tp→*F Tp		Tp→#		Tp→#
F			F→(E)		F→id	

6.2.3 Parse Trace (Observed)

Listing 6.3: Trace excerpt for “id + id * id”; match rows omitted for brevity as printed

```
Step  Stack          Input           Action
-----------------------------------------------------
1     $ E            id + id * id $  E -> T Ep
2     $ Ep T         id + id * id $  T -> F Tp
3     $ Ep Tp F      id + id * id $  F -> id
...   (match id; Tp -> #, Ep -> + T Ep; match +;
       T -> F Tp; F -> id; match id; Tp -> * F Tp;
       match *; F -> id; match id; Tp -> #; Ep -> #)
25    $              $               accept
Result: INPUT ACCEPTED
```

6.2.4 Syntax-Directed Translation (Observed)

Listing 6.4: Translation outputs as printed by the tool

```
Input "a + b * c":
  Postfix:              a b c * +
  Three-address code:   t1 = b * c
                        t2 = a + t1

Input "id + id * id":
  Postfix:              id id id * +
  Three-address code:   t1 = id * id
                        t2 = id + t1
```

The precedence behaviour is faithful (multiplication binds tighter because `Tp -> * F Tp` reduces *first* in the post-order emission), and parentheses re-associate the emitted code exactly as the grammar dictates (verified in the suite for `( a + b ) * ( a + c )` yielding `t3 = t1 * t2`). The demonstration input “`id + id * id`” is

one expression; confirming that this example parses does not, by itself, prove that *every* possible grammar or input is handled correctly — that assurance rests on the pattern of 26 checks in Annexure 1, each of which isolates one behaviour.

6.3 ERROR HANDLING

Error handling was exercised as deliberately as the success path, with each failure mode producing diagnostics at the stage that detects it:

- **Grammar stage.** Eight malformed corpora (T06a–T06f plus variants) are each rejected with a named reason and the offending line number; the pipeline refuses to emit any downstream artefact on any rejection.
- **Analysis stage.** FIRST/FOLLOW/table stages re-verify their preconditions ("not validated / mismatched input") and capacity limits before filling the table.
- **Parser stage.** Positioned diagnostics carry the token position, the offending token, the exact missing cell (`No parsing-table entry for [T, *]`) and the row's expected terminals; trailing garbage after acceptance is reported distinctly; step and stack caps prevent any non-termination.
- **Translation stage.** Semantic stack overflow/underflow and temporary/line caps surface as lastError in the translation result rather than terminating the process.

No input in the corpus crashes the tool; every buffer write is bounds checked against the centralised limits of Table 4.4.

6.4 DISCUSSION: COMPLEXITY AND QUALITY OF RESULTS

The stated asymptotics distinguish *theoretical estimates* from *executed behaviour*:

- **FIRST/FOLLOW.** The number of sweeps is bounded by $|V_T|+1$ (lattice height, Section 3.4); at course scale the observed sweep counts were 2–3. Per sweep the cost is $O(P \cdot R \cdot c)$ where c is the per-substring FIRST evaluation cost; no recursion is used, so recursive grammars behave identically to

acyclic ones.

- **Table construction.** $O(P \cdot T)$ placements with $O(T)$ column lookup (linear scan); seconds for the limit envelope (50×51 cells).
- **Parsing.** Each step consumes one lookahead or one stack symbol; steps are bounded by $O(|x|)$ for LL(1) grammars, and the observed trace for the demo input was 25 steps including matches.
- **Translation.** Produces the postfix string during the parse and evaluates it for TAC in $O(|\text{postfix}|)$.

No wall-clock benchmarking claims are made: inputs at course scale execute effectively instantaneously, and no performance measurement beyond the step counts above was recorded. *Theoretical* estimates only are presented for larger grammars, as honest estimates subject to the stated assumptions (bounded set sizes, terminal-order lookups).

6.5 SUMMARY OF THE RESULTS

The executed evidence supports one sentence per objective mapped in Section 1.4: the pipeline accepts arbitrary documented grammars, computes theory-consistent FIRST/FOLLOW sets, builds a correct LL(1) table, reports non-LL(1) grammars with exact conflict evidence, parses inputs with a visible linear-time trace and precise diagnostics, and produces correct postfix and three-address code — all under a test suite of 26 automated checks whose observed outcome was **26 passed, 0 failed**.

CHAPTER 7

CONCLUSION, LIMITATIONS AND SCOPE FOR FUTURE WORK

7.1 CONCLUSION

This project set out to remove the manual stage of LL(1) parser construction — hand-computed FIRST sets, hand-copied predictive tables and hand-written grammar-specific loops — and replace it with a grammar-agnostic, auditable tool. The delivered system meets that aim:

- **Automated LL(1) parser generation.** Any grammar in the documented line format flows through validation, classification, FIRST, FOLLOW, table construction, conflict detection, parsing and translation without a single grammar-specific line of code; the demonstration results of Chapter 6 were produced by the pipeline, not inserted into it.
- **Theory-faithful FIRST/FOLLOW computation.** The sets are computed as least fixed points of the classical equations, with epsilon propagation and direct and indirect recursion verified by executed tests.
- **Predictive parsing-table generation with conflict evidence.** Cells store production references; a claimed-twice cell is reported with both productions and the terminal, and parsing is refused for non-LL(1) grammars.
- **Predictive parsing with full observability.** The stack/input/action trace, the '$' end-marker discipline, positioned diagnostics and bounded resources were all exercised by the executed suite.
- **Syntax-directed translation.** Postfix output and three-address code are emitted from parser events on the same parse, demonstrating event-driven SDT without a separate pass.
- **Verified, reproducible quality.** The 26-check automated suite was executed with the observed result "Tests passed: 26, failed: 0", covering valid grammars, invalid productions, epsilon handling, conflicts, trace/diagnostic be-

haviour and both translation outputs.

Beyond the functional objectives, the project's second-order lesson is methodological: centralising every capacity in one header, paralleling each module with its test corpus, and refusing to proceed on invalid input are what made the system simultaneously simple to reason about and robust in use.

7.2 CONTRIBUTIONS OF THE TEAM

Each member owned, implemented and can explain one module end to end — grammar processing and validation (24BCE0092), FIRST computation (24BDE0155), FOLLOW computation (24BCE0556), parsing-table construction with conflict detection (24BCI0103), and the predictive parser with syntax-directed translation (24BDS0044) — while the integration driver, the test suite, the documentation and the review material were produced jointly.

7.3 LIMITATIONS

The following limitations are stated as properties of the delivered system, not as possibilities:

1. **Fixed capacities.** All buffers are static arrays bounded by Table 4.4; very large grammars or inputs are rejected safely rather than handled.
2. **No grammar transformation.** Left-recursive or left-factoring-required grammars are reported NOT LL(1) with their conflicting cells; the tool does not rewrite them.
3. **No lexer layer.** Input tokens are whitespace-separated words; multi-character terminals like `id` are just words, and no regular-expression specification is supported.
4. **SDT shape rules.** The built-in translation covers the standard operator-encoding pattern; RHS shapes outside it translate as plain concatenation (a documented echo) rather than failing.
5. **Trace and semantic caps.** Extremely long inputs truncate the printed trace (with a notice) or stop the translation with an error at the documented limits.

6. **LL(1) only.** No conflict-resolution heuristics and no LR fallback; SLR/LALR behaviour is out of scope.
7. **Implicit '$' column.** The end-marker column is functional but not printed as part of the terminal row.

7.4 SCOPE FOR FUTURE WORK

The architecture makes each enhancement a bounded, local change:

1. **Grammar transformation stage:** automatic left-recursion elimination and left factoring before table construction, with a before/after diff shown to the user.
2. **Richer diagnostics:** suggestions mode that names the offending alternative pair and the applicable LL(1) condition (the three-condition test) alongside the conflict report.
3. **Parse-tree visualisation:** the parser's reduction history already encodes a leftmost derivation; rendering it as a tree would complete the classical toolset.
4. **Additional translation schemes:** infix-to-prefix, syntax tree construction and typed TAC emission behind the existing event hook.
5. **Formats and ergonomics:** multi-line alternatives, comments in grammar files, and a library mode exposing the modules for classroom experiments.
6. **Dynamic storage normalization:** replacing the largest static buffers with arena allocation to lift the capacity limits while keeping the bounds-checking discipline.

CHAPTER A

TEST CASES

This annexure lists the automated integration suite (`testcases/run_tests.sh`) in the standard test-case format. All cases are executed by one suite run whose bottom line is printed after the register; the individual execution date should be recorded in Table 5.1 (Chapter 5) when you re-verify on your machine.

A.1 TEST CASE TEMPLATE

Table A.1: Test case template (per-check)

Field	**Content**
Test Case ID	Txx (Txxb: sub-run of the same grammar)
Test Case Description	One-sentence intent of the scenario
Test Case Type	Integration (black-box, via the CLI binary)
Pre-conditions	Executable built; testcases/ tree present
Input	Grammar file path and/or stdin strings
Expected / Actual Result	Marker strings the suite greps for
Pass/Fail	Status of the most recent executed run

A.2 EXECUTED TEST CASE REGISTER

Table A.2: Complete executed test-case register (26 checks)

ID	Description	Input	Expected marker	Status
T01	Demo grammar end-to-end	expression_valid.txt	FIRST/FOLLOW exact sets; “Grammar is LL(1)”	PASS
T02	Epsilon propagation	epsilon_propagation.txt	FIRST(S)={a ,b}; nullable A,B	PASS
T03	Pure-epsilon grammar	pure_epsilon.txt	FIRST(S)={#}; LL(1)	PASS
T03b	Epsilon input string	empty line on pure_epsilon	INPUT ACCEPTED	PASS
T04	Conflict detection	non_ll1_conflict.txt	“Conflict detected at M[S, a]”; NOT LL(1)	PASS
T05	Minimal grammar	simple.txt (S→a)	FIRST(S)={a}; FOLLOW(S)={}	PASS
T05b	Accept valid input	“a”	INPUT ACCEPTED	PASS
T05c	Reject invalid input	“b”	SYNTAX ERROR; no entry [S,b]	PASS
T06a	Undefined non-terminal	undefined_nt.txt	rejected: “never defined”	PASS
T06b	Missing arrow	missing_arrow.txt	rejected: “missing ‘->’ ”	PASS
T06c	Double arrow	double_arrow.txt	rejected: “more than one arrow”	PASS
T06d	Invalid symbol	bad_symbol.txt (a$b)	rejected: “invalid symbol”	PASS
T06e	Empty alternative	empty_alt.txt	rejected: “empty alternative”	PASS
T06f	End marker in grammar	end_marker.txt	rejected: ‘$’ reserved	PASS
T07	Indirect recursion	indirect_recursion.txt	correctly NOT LL(1) (M[A,c] clash)	PASS

ID	Description	Input	Expected marker	Status
T08	Valid input + trace	demo grammar, 3 strings	INPUT ACCEPTED with full trace	PASS
T09	Invalid input (mid)	“id + * id”	SYNTAX ERROR no entry [T,*]	PASS
T09b	Invalid input (end)	“id +”	SYNTAX ERROR	PASS
T10	SDT operators	“a + b * c”	postfix “a b c * +”; TAC t1,$t2$	PASS
T10b	SDT variant	“id * id + id”	postfix “id id * id +”; TAC t1, t2	PASS
T11	Compact arrow syntax	no_space_arrow.txt	accepted; LL(1)	PASS
T12	Nullable in RHS	optional_symbol.txt	accepted; LL(1)	PASS
T12b	Optional pars-ing	“a b” and “b”	both ACCEPTED	PASS
T13	Left recursion	left_recursive.txt	NOT LL(1) reported	PASS

Final line of the executed run:

```
Tests passed: 26   failed: 0
```

CHAPTER B

CODE

The complete source (2 976 lines across 8 modules, plus headers, tests and documentation) is delivered in the project repository. This annexure reproduces the three excerpts that carry the intellectual core of the system, with the files referenced. *(The listing line numbers refer to the delivered files.)*

B.1 FIRST FIXED-POINT SWEEP (SRC/FIRST.C — EXCERPT)

Listing B.1: FIRST fixed-point iteration (abridged from src/first.c)

```
int computeAllFirstSets(Grammar *g, FirstCollection *out) {
    initializeFirstCollection(out, g);
    int changed = 1;
    while (changed) {                       /* least fixed point   */
        changed = 0;
        for (int p = 0; p < g->productionCount; ++p) {
            Production *pr = &g->productions[p];
            for (int a = 0; a < pr->rhsCount; ++a) {
                if (productionAltIsEpsilon(pr->rhs[a])) {
                    addEpsilonToFirstSet(out, p);   /* A -> #
                    changed |= 1;
                } else {
                    char sym[MAX_SYMBOL_LENGTH];
                    int nullable = 1;
                    int cnt = productionAltSymbolCount(pr->rhs[a]);
                    for (int i = 0; i < cnt; ++i) {
                        productionAltGetSymbol(pr->rhs[a], i, sym);
                        addSymbolsToFirstSet(out, p, sym, g);
                        if (!firstSetContainsEpsilon(...)) {
                            nullable = 0; break;    /* stop the chain */
                        }
                    }
                    if (nullable) addEpsilonToFirstSet(out, p);
```

```
                }
            }
        }
    }
    return 1;
}
```

(Identifiers are lightly abridged; the delivered file is normative.)

B.2 FOLLOW RULES SWEEP (SRC/FOLLOW.C — EXCERPT)

Listing B.2: FOLLOW fixed point: Rules 1–3 (abridged from src/follow.c)

```
int initializeFollowCollection(void);
/* Rule 1: $ belongs to FOLLOW(start). */
addTerminalToFollowSet(follows, startIndex, END_MARKER, g);

while (changed) {
    for (each production A -> alpha) {
        for (each symbol position i of alpha) {
            if (symbol is non-terminal B) {
                beta = suffix of alpha after B;
                firstOfSequence(beta, g, firsts, seq, &eps);
                /* Rule 2: FIRST(beta)\{#} into FOLLOW(B). */
                addSequenceSymbolsExceptEpsilon(...);
                /* Rule 3: beta nullable (or empty) => FOLLOW(A). */
                if (betaEmpty || eps)
                    mergeFollowInto(follows, B_index, A_index, g);
            }
        }
    }
}
```

B.3 PREDICTIVE PARSE LOOP (SRC/PARSER.C — EXCERPT)

Listing B.3: Table-driven predictive loop (abridged from src/parser.c)

```
/* stack = "$ start"; tokens already end with "$". */
```

```
while (!accepted && !state.lastError) {
    tracePrintRow(&state, ...);
    if (++state.stepNumber > MAX_PARSE_STEPS)
        { lastError = "step limit exceeded"; break; }
    const char *X = topSymbol(&state);
    const char *a = curToken(&state);
    if (isEndMarker(X)) {
        if (isEndMarker(a)) accepted = 1;             /* accept   */
        else trailingGarbageError(&state, a);          /* $ vs a   */
    } else if (grammarIsTerminal(g, X)) {
        if (matchToken(X, a)) { fireShift(X); pop(&state); advance(&state);
            }
        else mismatchError(&state, X, a);
    } else {
        TableEntry e = tableLookup(table, X, a);
        if (isEmptyEntry(&e)) noTableEntryError(&state, X, a);  /* +expect
            */
        else { popNonTerminal(&state);
               pushRhsReversed(&state, e.lhsIndex, e.altIndex);
               recordReduceMarker(e.lhsIndex, e.altIndex); }
    }
}
```

B.4 GRAMMAR CORPORA SAMPLE

The three representative corpora of the suite (`testcases/valid/expression_valid.txt`, `testcases/invalid/missing_arrow.txt`, `testcases/edge/pure_epsilon.txt`):

Listing B.4: Sample grammar corpora

```
# valid/expression_valid.txt
E  -> T Ep
Ep -> + T Ep | #
T  -> F Tp
Tp -> * F Tp | #
F  -> ( E ) | id

# invalid/missing_arrow.txt   (line 1 has no "->")
E T Ep
```

```
# edge/pure_epsilon.txt
S -> #
```

CHAPTER C

REFERENCES

BIBLIOGRAPHY

[1] P. M. Lewis, II and R. E. Stearns, “Syntax-directed transduction,” *Journal of the ACM*, vol. 15, no. 3, pp. 465–488, Jul. 1968, doi: 10.1145/321466.321477.

[2] D. J. Rosenkrantz and R. E. Stearns, “Properties of deterministic top-down grammars,” *Information and Control*, vol. 17, no. 3, pp. 226–256, 1970, doi: 10.1016/S0019-9958(70)90446-8.

[3] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Pearson/Addison-Wesley, 2006, ISBN 978-0-321-48681-3.

[4] A. V. Aho and J. D. Ullman, *The Theory of Parsing, Translation, and Compiling*, vol. 1: *Parsing*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972.

[5] D. Grune and C. J. H. Jacobs, *Parsing Techniques: A Practical Guide*, 2nd ed. London, U.K.: Springer, 2008, ISBN 978-1-4419-1901-4.

[6] S. C. Johnson, *Yacc: Yet Another Compiler-Compiler*, Computing Sciences Technical Report No. 32, Bell Laboratories, Murray Hill, NJ, USA, Jul. 1975.

[7] Free Software Foundation, *GNU Bison: The Yacc-Compatible Parser Generator*, ver. 3.8.1, Sep. 2021. [Online]. Available: `https://www.gnu.org/software/bison/manual/`

[8] T. Parr, *The Definitive ANTLR 4 Reference*. Raleigh, NC, USA: Pragmatic Bookshelf, 2013, ISBN 978-1-934356-99-9.

[9] ISO/IEC, *Programming Languages — C*, ISO/IEC 9899:1999, 2nd ed., Geneva, Switzerland, Dec. 1999.

[10] ISO/IEC, *Information Technology — Programming Languages — C*, ISO/IEC 9899:2011, 3rd ed., Geneva, Switzerland, Dec. 2011.